



TLB & Demand Paging

Week 9: Hit, Miss, dan Page Fault

informatika-itera.github.io/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

OUTLINE

Peta materi pertemuan ini

1. **Review & Motivasi**
2. **TLB: Translation Lookaside Buffer**
3. **Demand Paging**
4. **Page Replacement**
5. **Latihan Terpadu**
6. **Ringkasan**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

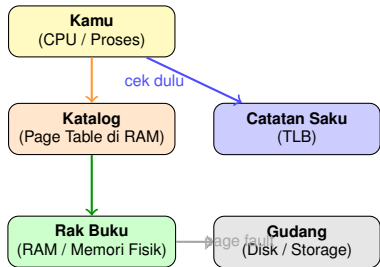
1. Menjelaskan mengapa page table biasa menyebabkan overhead akses memori.
2. Memahami cara kerja TLB dan membedakan TLB hit vs TLB miss.
3. Menghitung *Effective Access Time* (EAT) berdasarkan hit rate TLB.
4. Memahami konsep demand paging dan alur penanganan page fault.
5. Membandingkan algoritma page replacement: FIFO, Optimal, dan LRU.

Fokus Kelas Ini

Kita fokus pada **cara kerja TLB**, **perhitungan EAT**, dan **algoritma page replacement** secara step-by-step.

Analogi: Perpustakaan & Catatan Saku

Bayangkan kamu sedang mencari buku di perpustakaan...



- **Katalog** = Page table: tahu lokasi semua buku, tapi harus didatangi dulu (lambat).
- **Catatan saku (TLB)** = Kamu tulis lokasi buku yang **sering dicari** — cek sini dulu sebelum ke katalog!
- **Rak buku (RAM)** = Buku yang sudah ada di depanmu, langsung bisa dibaca.
- **Gudang (Disk)** = Buku belum di rak → harus diambil dulu, **butuh waktu lama**.

Review: Paging & Page Table

Rekap singkat dari Week 7

Yang sudah kita pelajari:

- Memori logis dibagi menjadi **page**, memori fisik dibagi menjadi **frame**.
- **Page table** memetakan page number $\rightarrow$ frame number.
- Alamat logis = (page #, offset) $\rightarrow$ alamat fisik = (frame #, offset).
- **Valid bit** menunjukkan apakah page ada di memori.

Rumus Translasi

Alamat Logis: (p, d)

$\downarrow$ *lookup page table*

Frame number $f = PT[p]$

Alamat Fisik: (f, d)

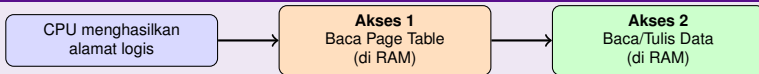
Pertanyaan

Seberapa **cepat** proses translasi ini? Apakah ada masalah?

Masalah: Setiap Akses Butuh 2x Baca RAM

Overhead page table pada sistem nyata

Alur Akses Memori dengan Page Table



Konsekuensinya:

- Setiap instruksi yang mengakses memori → **2 kali** baca RAM.
- Waktu akses efektif berlipat ganda.
- Semakin besar program, semakin besar page table-nya.

Ilustrasi

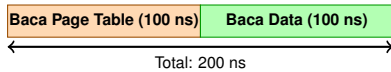
Jika akses RAM normal = **100 ns**,
maka dengan page table:
 $100 + 100 = \mathbf{200\ ns}$ per akses logis.

Program lambat $2\times$ hanya untuk translasi!

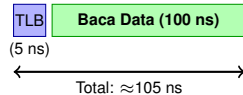
Visualisasi Overhead: Tanpa vs Dengan Optimasi

Perbandingan waktu akses memori

Tanpa Optimasi (Page Table Biasa)



Dengan TLB (Hit)



Kenapa Jauh Lebih Cepat?

- **TLB** adalah cache kecil dan **sangat cepat** (chip CPU).
- Menyimpan entri page table yang baru-baru ini dipakai.
- Jika ditemukan di TLB (*TLB hit*) → tidak perlu baca RAM untuk translasi.
- Program nyata: hit rate TLB bisa mencapai **95–99%**.

Solusi 1: TLB — Translation Lookaside Buffer

Cache untuk page table

Definisi

TLB adalah **cache asosiasif berkecepatan tinggi** yang menyimpan pasangan $\langle \text{page number}, \text{frame number} \rangle$ yang baru-baru ini digunakan.

TLB Hit (✓ cepat)

- Page number ditemukan di TLB.
- Frame number langsung didapat.
- **Tidak perlu** akses page table di RAM.
- Waktu: akses TLB + akses data.

TLB Miss (× lebih lambat)

- Page number **tidak** ada di TLB.
- Harus baca page table di RAM.
- Entri baru dimuat ke TLB untuk akses berikutnya.
- Waktu: akses TLB + akses page table + akses data.

Solusi 2: Demand Paging

Hanya muat page saat dibutuhkan

Definisi

Demand paging adalah strategi di mana page **tidak langsung dimuat ke RAM** saat proses dimulai, melainkan hanya dimuat **saat pertama kali diakses**.

Page Hit (✓ normal)

- Page yang diakses **sudah ada** di RAM.
- Valid bit = 1 → translasi berjalan normal.

Page Fault (× perlu penanganan)

- Page yang diakses **belum ada** di RAM.
- Valid bit = 0 → OS harus muat dari disk.
- Jauh lebih lambat (ms vs ns).

Keuntungan

- RAM tidak terbuang untuk page yang tidak pernah dipakai.
- Proses bisa berjalan meski ukurannya lebih besar dari RAM fisik.
- Dasar dari *virtual memory* modern.

Apa Itu TLB?

Cache asosiasif untuk mempercepat translasi alamat

Definisi

TLB (*Translation Lookaside Buffer*) adalah **cache berkecepatan tinggi** yang berada di dalam chip CPU, menyimpan pasangan $\langle \text{page number}, \text{frame number} \rangle$ yang baru-baru ini digunakan.

Karakteristik TLB:

- Berada di dalam atau dekat chip CPU.
- Kapasitas **sangat kecil**: 16 – 1024 entri.
- Kecepatan **sangat tinggi**: 0.5 – 5 ns.
- Pencarian dilakukan **secara paralel** (asosiatif), bukan sekuensial seperti array.

Isi setiap entri TLB:

Page #	Frame #
3	7
7	2
12	0
...	...

Entri lama diganti saat TLB penuh (*replacement policy*)^{2.0}.

Mekanisme: TLB Hit

Skenario terbaik — translasi tanpa akses RAM tambahan

Kondisi: Page number ditemukan di TLB



Langkah-langkah:

1. CPU menghasilkan alamat logis (p, d) .
2. TLB dicek: page p **ditemukan**.
3. Frame number f langsung diambil dari TLB.
4. Alamat fisik (f, d) digunakan untuk akses data.

Waktu Akses (TLB Hit)

$$T_{\text{hit}} = t_{\text{TLB}} + t_{\text{mem}}$$

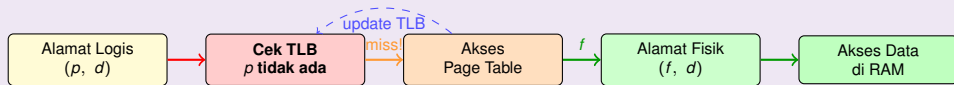
Contoh: $5 + 100 = \mathbf{105 \text{ ns}}$

Jauh lebih cepat dari 200 ns
tanpa TLB!

Mekanisme: TLB Miss

Skenario penalti — harus akses page table di RAM

Kondisi: Page number *tidak* ditemukan di TLB



Langkah-langkah:

1. CPU hasilkan alamat logis (p, d).
2. TLB dicek: p **tidak ditemukan** (miss).
3. Akses page table di RAM $\rightarrow$ dapat f .
4. Muat entri (p, f) ke TLB.
5. Akses data di RAM via (f, d).

Waktu Akses (TLB Miss)

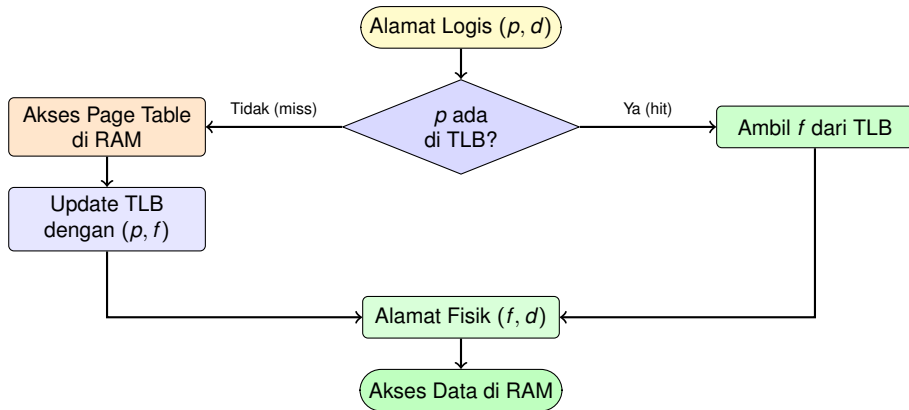
$$T_{\text{miss}} = t_{\text{TLB}} + 2 \cdot t_{\text{mem}}$$

Contoh: $5 + 200 = \mathbf{205 \text{ ns}}$

Lebih lambat $2\times$ dibanding TLB hit!

Diagram Alur: TLB Hit vs Miss

Perbandingan jalur eksekusi



Effective Access Time (EAT)

Mengukur waktu rata-rata akses memori dengan TLB

Rumus EAT

$$\text{EAT} = \alpha \cdot (t_{\text{TLB}} + t_{\text{mem}}) + (1 - \alpha) \cdot (t_{\text{TLB}} + 2 \cdot t_{\text{mem}})$$

Keterangan variabel:

- α = **TLB hit rate** (proporsi akses yang *hit*)
- t_{TLB} = waktu akses TLB
- t_{mem} = waktu akses RAM (1 kali)
- Suku pertama: kontribusi **TLB hit**
- Suku kedua: kontribusi **TLB miss** (butuh 2 kali akses RAM)

Sederhanakan

$$\text{EAT} = t_{\text{TLB}} + t_{\text{mem}} + (1 - \alpha) \cdot t_{\text{mem}}$$

Semakin tinggi α (hit rate), semakin kecil EAT dan semakin cepat sistem!

Contoh Perhitungan EAT

Sistem dengan hit rate 90% dan 99%

Diketahui: $t_{TLB} = 5 \text{ ns}$, $t_{mem} = 100 \text{ ns}$

Kasus A: $\alpha = 0.90$ (90%)

$$\begin{aligned} \text{EAT} &= 0.90 \times (5 + 100) \\ &\quad + 0.10 \times (5 + 200) \\ &= 0.90 \times 105 + 0.10 \times 205 \\ &= 94.5 + 20.5 \\ &= \mathbf{115 \text{ ns}} \end{aligned}$$

Kasus B: $\alpha = 0.99$ (99%)

$$\begin{aligned} \text{EAT} &= 0.99 \times (5 + 100) \\ &\quad + 0.01 \times (5 + 200) \\ &= 0.99 \times 105 + 0.01 \times 205 \\ &= 103.95 + 2.05 \\ &= \mathbf{106 \text{ ns}} \end{aligned}$$

Kesimpulan

Hit rate naik dari 90% ke 99% $\rightarrow$ EAT turun dari 115 ns ke 106 ns.

Tanpa TLB sama sekali: 200 ns. TLB **sangat signifikan** pengaruhnya!

Latihan: Hitung EAT

Kerjakan sebelum melihat solusi!

Soal

Sebuah sistem memiliki spesifikasi berikut:

- Waktu akses TLB: $t_{\text{TLB}} = 10 \text{ ns}$
- Waktu akses RAM: $t_{\text{mem}} = 150 \text{ ns}$
- TLB hit rate: $\alpha = 0.95$

Hitunglah **Effective Access Time (EAT)** sistem tersebut!

Rumus yang Digunakan

$$\text{EAT} = \alpha \cdot (t_{\text{TLB}} + t_{\text{mem}}) + (1 - \alpha) \cdot (t_{\text{TLB}} + 2 \cdot t_{\text{mem}})$$

Solusi Latihan EAT

Pembahasan langkah demi langkah

Diketahui: $t_{\text{TLB}} = 10 \text{ ns}$, $t_{\text{mem}} = 150 \text{ ns}$, $\alpha = 0.95$

Langkah 1 — Hitung T_{hit} :

$$T_{\text{hit}} = t_{\text{TLB}} + t_{\text{mem}} = 10 + 150 = 160 \text{ ns}$$

Langkah 2 — Hitung T_{miss} :

$$T_{\text{miss}} = t_{\text{TLB}} + 2 \cdot t_{\text{mem}} = 10 + 300 = 310 \text{ ns}$$

Langkah 3 — Hitung EAT:

$$\begin{aligned} \text{EAT} &= 0.95 \times 160 + 0.05 \times 310 \\ &= 152 + 15.5 \\ &= \mathbf{167.5 \text{ ns}} \end{aligned}$$

Perbandingan

Kondisi	Waktu
Tanpa TLB	300 ns
EAT ($\alpha = 0.95$)	167.5 ns
TLB hit penuh	160 ns

TLB menghemat $\approx 44\%$ waktu akses dibanding tanpa TLB!

TLB dan Context Switch

Apa yang terjadi saat proses berganti?

Masalah

TLB menyimpan entri milik proses yang sedang berjalan. Saat **context switch** terjadi, entri TLB dari proses lama **tidak berlaku** untuk proses baru!

Solusi 1: TLB Flush

- Seluruh isi TLB dikosongkan saat context switch.
- Sederhana, tapi proses baru mulai dari **miss semua**.
- Perlu waktu untuk mengisi ulang TLB.

Solusi 2: ASID (Address Space ID)

- Setiap entri TLB diberi tag **ID proses (ASID)**.
- TLB bisa menyimpan entri dari **banyak proses** sekaligus.
- Hanya entri dengan ASID yang cocok yang dianggap valid.
- Lebih efisien, tidak perlu flush penuh.

Checkpoint Pemahaman: TLB

Cek sebelum lanjut

1. Apa perbedaan antara TLB hit dan TLB miss? Mana yang lebih cepat dan kenapa?
2. Jika hit rate TLB = 80%, $t_{\text{TLB}} = 2 \text{ ns}$, $t_{\text{mem}} = 100 \text{ ns}$, berapa EAT-nya?
3. Mengapa TLB perlu di-*flush* saat context switch terjadi?
4. Apa keuntungan menggunakan ASID dibandingkan TLB flush biasa?
5. Jika TLB penuh dan ada entri baru yang harus dimasukkan, apa yang terjadi?

Jawaban Singkat

Jika Anda bisa menjawab semua dengan percaya diri, Anda siap untuk Bagian 3: Demand Paging!

Apa Itu Demand Paging?

Muat page hanya saat benar-benar dibutuhkan

Definisi

Demand paging adalah strategi di mana page **tidak dimuat ke RAM saat proses dimulai**, melainkan hanya dimuat **saat pertama kali diakses** oleh proses.

Tanpa Demand Paging:

- Semua page proses dimuat ke RAM saat start.
- Boros memori — banyak page yang tidak pernah diakses.
- Proses besar tidak bisa berjalan jika RAM tidak cukup.

Dengan Demand Paging:

- Hanya page yang **dibutuhkan** yang ada di RAM.
- RAM lebih efisien digunakan.
- Proses bisa berjalan meski ukurannya **melebihi RAM fisik**.
- Dasar dari *virtual memory* modern.

Valid/Invalid Bit

Penanda apakah page sedang ada di RAM atau tidak

Format Page Table dengan Valid Bit

Page #	Frame #	V/I	Status
0	4	V	Ada di RAM
1	—	I	Tidak di RAM
2	7	V	Ada di RAM
3	—	I	Tidak di RAM
4	1	V	Ada di RAM

Arti bit:

- **V (Valid = 1):** Page ada di RAM, translasi bisa dilanjutkan.
- **I (Invalid = 0):** Page **tidak ada di RAM** — terjadi *page fault*, OS harus turun tangan.

Perhatian

Invalid bukan berarti alamat salah! Bisa jadi page-nya memang valid tapi belum dimuat ke RAM.

Page Hit — Alur Akses Normal

Ketika page yang diakses sudah ada di RAM

Kondisi: Valid bit = 1



Langkah-langkah:

1. CPU hasilkan alamat logis (p, d).
2. Cek page table: valid bit = **V**.
3. Frame number f diperoleh.
4. Akses data di RAM via (f, d).

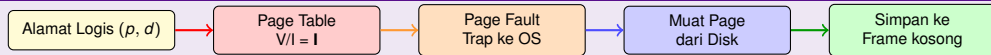
Catatan

Page hit adalah skenario **normal dan cepat**. Tidak ada intervensi OS — semua ditangani oleh hardware (MMU).

Page Fault — Alur Penanganan

Ketika page yang diakses belum ada di RAM

Kondisi: Valid bit = 0 → OS mengambil alih



Langkah-langkah:

1. MMU deteksi valid bit = 0 → *page fault trap*.
2. OS simpan state proses yang sedang berjalan.
3. OS cari frame kosong di RAM.
4. OS muat page dari disk ke frame tersebut.
5. Update page table: set valid bit = V, isi frame #.
6. Restart instruksi yang tadi gagal.

Mengapa Lambat?

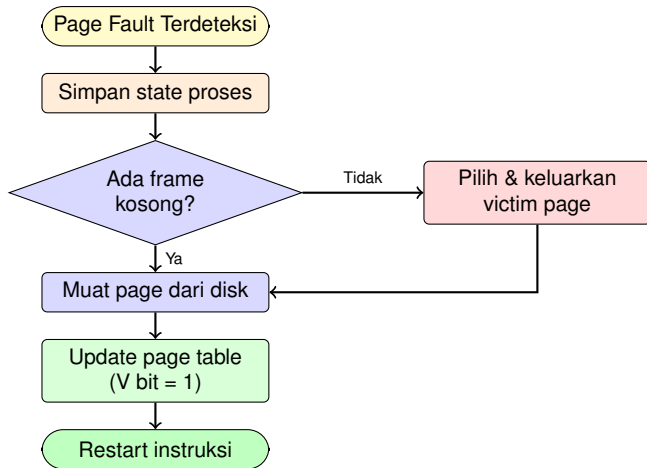
Akses disk jauh lebih lambat dari RAM:

- RAM: ~ 100 ns
- SSD: ~ 100.000 ns
- HDD: $\sim 10.000.000$ ns

Page fault → penalti **besar**!

Diagram Alur: Page Fault Handler

Langkah OS dari deteksi hingga resume proses



Contoh: Trace Akses Memori

Simulasi urutan akses dengan page fault

Kondisi awal: 3 frame kosong. Urutan akses page: **1, 2, 3, 1, 4, 1**

Ke-	Page	F0	F1	F2	Fault?
1	1	P.1	—	—	Ya
2	2	P.1	P.2	—	Ya
3	3	P.1	P.2	P.3	Ya
4	1	P.1	P.2	P.3	Tidak
5	4	P.4	P.2	P.3	Ya
6	1	P.4	P.2	P.1	Ya

Keterangan:

- Hijau : fault, frame kosong
- Merah : fault, ganti page lama
- Oranye : fault, dimuat ulang
- Tanpa warna: page hit

Hasil

Total akses: 6

Page fault: 5

Page hit: 1

(akses ke-4, page 1)

Pure Demand Paging vs Pre-paging

Dua pendekatan dalam memuat page ke memori

Pure Demand Paging

- Page **hanya dimuat** saat terjadi page fault.
- Tidak ada page di RAM saat proses dimulai.
- Hemat memori, tapi banyak page fault di awal (*cold start*).
- Cocok untuk proses yang besar dengan pola akses tidak menentu.

Pre-paging

- Beberapa page dimuat **sebelum dibutuhkan** (prediksi).
- Mengurangi page fault di awal.
- Bisa boros jika prediksi salah.
- Cocok jika pola akses dapat diprediksi (misal: sequential).

Checkpoint Pemahaman: Demand Paging

Cek sebelum lanjut

1. Apa perbedaan antara page hit dan page fault? Mana yang lebih cepat dan kenapa?
2. Apa fungsi valid/invalid bit dalam page table?
3. Sebutkan 6 langkah yang dilakukan OS saat menangani page fault!
4. Mengapa page fault sangat mahal (lambat) dibanding page hit?
5. Apa perbedaan antara pure demand paging dan pre-paging?

Jawaban Singkat

Jika Anda bisa menjawab semua dengan percaya diri, Anda siap untuk Bagian 4: Page Replacement!

Masalah: Frame Penuh

Harus ganti page mana saat semua frame terisi?

Situasi

Page fault terjadi, tapi **tidak ada frame kosong**. OS harus memilih satu page yang ada di RAM untuk **dikeluarkan** (*victim page*) agar framenya bisa dipakai.

Alur lengkap dengan replacement:

1. Page fault terdeteksi.
2. Tidak ada frame kosong.
3. OS pilih *victim page* berdasarkan algoritma.
4. Tulis victim ke disk jika *dirty* (*swap out*).
5. Muat page baru dari disk (*swap in*).
6. Update page table, restart instruksi.

Tujuan Algoritma Replacement

Memilih victim page yang **paling jarang dibutuhkan** di masa depan, sehingga page fault seminimal mungkin.

Tiga algoritma utama:

- FIFO (*First In, First Out*)
- Optimal (OPT)
- LRU (*Least Recently Used*)

Konsep: Victim Page & Dirty Bit

Detail teknis saat mengganti page

Victim Page

Page yang dipilih untuk **dikeluarkan** dari RAM. Dipilih oleh algoritma page replacement.

Dirty Bit (Modified Bit)

Bit yang menandakan apakah page **telah dimodifikasi** sejak terakhir dimuat.

- **Dirty = 0:** Sama dengan salinan di disk → langsung buang.
- **Dirty = 1:** Sudah berubah → **harus ditulis ke disk** dulu.

Mengapa Dirty Bit Penting?

Menulis ke disk sangat lambat. Dengan dirty bit, OS bisa **skip penulisan yang tidak perlu** jika page tidak dimodifikasi.

Page	Frame	Valid	Dirty
0	3	V	0
1	7	V	1
2	—	I	0

Algoritma FIFO

Ganti page yang paling lama berada di RAM

Prinsip

Keluarkan page yang **pertama kali masuk** ke RAM. Page yang paling lama di memori dijadikan victim.

Contoh: 3 frame, urutan akses: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3**

Akses	7	0	1	2	0	3	0	4	2	3
F0	7	7	7	2	2	2	2	4	4	4
F1	–	0	0	0	0	3	3	3	2	2
F2	–	–	1	1	1	1	1	1	1	3
Fault?	Ya	Ya	Ya	Ya	–	Ya	–	Ya	Ya	Ya

- Hijau : fault, frame kosong.
- Merah : fault, ganti page lama.
- Tanpa warna: page hit.

Hasil FIFO

Page fault: 9 dari 10.

Sederhana tapi sering tidak optimal.

Belady's Anomaly

Fenomena aneh pada algoritma FIFO

Anomali Belady

Pada algoritma FIFO, **menambah jumlah frame** terkadang justru **meningkatkan** jumlah page fault — berlawanan dengan intuisi!

Contoh dengan urutan akses: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

3 Frame → 9 page fault

Frame selalu penuh dan sering tergantikan karena ruang terbatas.

4 Frame → 10 page fault

Lebih banyak frame, tapi urutan penggantian FIFO menjadi kurang beruntung — lebih banyak fault!

Implikasi

Belady's anomaly hanya terjadi pada FIFO. Algoritma **OPT dan LRU tidak mengalami anomali** ini karena termasuk algoritma *stack-based*.

Algoritma Optimal (OPT)

Ganti page yang paling lama tidak akan digunakan

Prinsip

Keluarkan page yang **paling jauh dibutuhkan di masa depan**. Menghasilkan page fault paling sedikit, tapi **tidak bisa diimplementasikan** di sistem nyata (butuh pengetahuan masa depan). Digunakan sebagai acuan ideal.

Contoh: 3 frame, urutan akses: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3**

Akses	7	0	1	2	0	3	0	4	2	3
F0	7	7	7	2	2	2	2	2	2	2
F1	—	0	0	0	0	0	0	4	4	3
F2	—	—	1	1	1	3	3	3	3	3
Fault?	Ya	Ya	Ya	Ya	—	Ya	—	Ya	—	—

Victim dipilih dengan melihat ke depan: ganti page yang *next use*-nya paling jauh.

Hasil OPT

Page fault: 6 dari 10.
Lebih baik dari FIFO (9)!

Algoritma LRU

Ganti page yang paling lama tidak digunakan

Prinsip

Keluarkan page yang **terakhir kali digunakan paling lama yang lalu**. LRU menggunakan history penggunaan sebagai aproksimasi masa depan dan **bisa diimplementasikan** di sistem nyata.

Contoh: 3 frame, urutan akses: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3**

Akses	7	0	1	2	0	3	0	4	2	3
F0	7	7	7	2	2	2	2	4	4	3
F1	–	0	0	0	0	3	3	3	3	3
F2	–	–	1	1	1	1	1	1	2	2
Fault?	Ya	Ya	Ya	Ya	–	Ya	–	Ya	Ya	Ya

Victim dipilih berdasarkan timestamp: ganti page yang terakhir dipakai paling lama lalu.

Hasil LRU

Page fault: 8 dari 10.

Lebih baik dari FIFO, mendekati OPT.

Perbandingan: FIFO vs OPT vs LRU

Ringkasan tiga algoritma pada contoh yang sama

Urutan akses: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3** | Frame: 3

Algoritma	Fault	Hit	Fault Rate	Catatan
FIFO	9	1	90%	Sederhana, rentan Belady's anomaly
Optimal (OPT)	6	4	60%	Terbaik, tidak realistis
LRU	8	2	80%	Praktis, mendekati optimal

Kesimpulan

- **OPT** = batas bawah ideal (acuan).
- **LRU** = pilihan terbaik yang implementable.
- **FIFO** = paling mudah, tapi suboptimal.

Di Sistem Nyata

LRU murni mahal (perlu timestamp tiap akses). OS modern menggunakan aproksimasi seperti **Clock Algorithm** atau **NFU**.

Latihan: Bandingkan Ketiga Algoritma

Hitung page fault untuk FIFO, OPT, dan LRU

Soal

Diberikan **4 frame** (awalnya kosong) dan urutan akses page:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Untuk masing-masing algoritma (**FIFO**, **OPT**, **LRU**):

1. Trace isi frame setiap langkah.
2. Tandai setiap page fault.
3. Hitung total page fault.

Petunjuk

- **FIFO**: Ganti page yang paling lama masuk (antrian).
- **OPT**: Ganti page yang berikutnya dipakai paling jauh ke depan.
- **LRU**: Ganti page yang terakhir dipakai paling lama ke belakang.

Latihan 1: Hitung EAT

Kerjakan sebelum melihat solusi!

Soal

Sebuah sistem memori memiliki spesifikasi berikut:

- Waktu akses TLB: $t_{\text{TLB}} = 20 \text{ ns}$
- Waktu akses RAM: $t_{\text{mem}} = 200 \text{ ns}$
- TLB hit rate: $\alpha = 0.80$

- (a) Hitung EAT sistem tersebut.
(b) Berapa EAT jika hit rate ditingkatkan menjadi 95%?
(c) Berapa EAT tanpa TLB sama sekali?

Rumus

$$\text{EAT} = \alpha \cdot (t_{\text{TLB}} + t_{\text{mem}}) + (1 - \alpha) \cdot (t_{\text{TLB}} + 2 \cdot t_{\text{mem}})$$

Solusi Latihan 1

Pembahasan (a), (b), dan (c)

Diketahui: $t_{TLB} = 20 \text{ ns}$, $t_{mem} = 200 \text{ ns}$

(a) $\alpha = 0.80$:

$$T_{hit} = 20 + 200 = 220 \text{ ns}$$

$$T_{miss} = 20 + 400 = 420 \text{ ns}$$

$$\begin{aligned} EAT &= 0.80 \times 220 + 0.20 \times 420 \\ &= 176 + 84 = \mathbf{260 \text{ ns}} \end{aligned}$$

(b) $\alpha = 0.95$:

$$\begin{aligned} EAT &= 0.95 \times 220 + 0.05 \times 420 \\ &= 209 + 21 = \mathbf{230 \text{ ns}} \end{aligned}$$

(c) Tanpa TLB:

$$EAT = 2 \times t_{mem} = \mathbf{400 \text{ ns}}$$

Perbandingan

Kondisi	EAT
Tanpa TLB	400 ns
TLB ($\alpha=80\%$)	260 ns
TLB ($\alpha=95\%$)	230 ns
TLB hit penuh	220 ns

TLB 80% sudah hemat **35%** dibanding tanpa TLB. Naik ke 95% hemat **42.5%**!

Latihan 2: FIFO vs LRU

Trace page replacement dan bandingkan hasilnya

Soal

Diberikan **3 frame** (awalnya kosong) dan urutan akses page:

2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Untuk algoritma **FIFO** dan **LRU**:

1. Trace isi frame setiap langkah.
2. Tandai setiap page fault.
3. Hitung total page fault masing-masing.
4. Tentukan mana yang lebih efisien.

Petunjuk

- **FIFO**: Catat urutan masuknya page. Ganti yang paling pertama masuk.
- **LRU**: Catat kapan terakhir page diakses. Ganti yang paling lama tidak dipakai.

Solusi Latihan 2: FIFO

Urutan akses: 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Akses	2	3	2	1	5	2	4	5	3	2	5	2
F0	2	2	2	2	5	5	5	5	3	3	3	3
F1	–	3	3	3	3	3	4	4	4	2	2	2
F2	–	–	–	1	1	1	1	1	1	1	5	5
Fault?	Ya	Ya	–	Ya	Ya	–	Ya	–	Ya	Ya	Ya	–

- Hijau : fault, frame kosong.
- Merah : fault, ganti page lama.
- Tanpa warna: page hit.

Hasil FIFO

Page fault: 9 dari 12 akses

Page hit: 3 (akses ke 3, 6, 12)

Solusi Latihan 2: LRU

Urutan akses: 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Akses	2	3	2	1	5	2	4	5	3	2	5	2
F0	2	2	2	2	2	2	4	4	3	2	2	2
F1	–	3	3	3	5	5	5	5	5	5	5	5
F2	–	–	–	1	1	1	1	1	1	1	5	5
Fault?	Ya	Ya	–	Ya	Ya	–	Ya	–	Ya	Ya	Ya	–

Hasil LRU

Page fault: 9 dari 12 akses

Page hit: 3 (akses ke 3, 6, 12)

Kesimpulan

Pada kasus ini FIFO dan LRU menghasilkan jumlah page fault yang **sama (9)**. Hasilnya bisa berbeda tergantung urutan akses dan jumlah frame.

Ringkasan: Poin Kunci

Intisari materi pertemuan ini

TLB & EAT:

1. Page table biasa butuh **2 akses RAM** per translasi.
2. TLB menyimpan entri terbaru secara **asosiatif dan cepat**.
3. **TLB hit** $\rightarrow t_{\text{TLB}} + t_{\text{mem}}$.
4. **TLB miss** $\rightarrow t_{\text{TLB}} + 2t_{\text{mem}}$.
5. $\text{EAT} = \alpha \cdot T_{\text{hit}} + (1 - \alpha) \cdot T_{\text{miss}}$.
6. Context switch $\rightarrow$ TLB flush atau ASID.

Demand Paging & Page Replacement:

1. Valid bit = 0 $\rightarrow$ **page fault**, OS mengambil alih.
2. Page fault mahal: harus akses disk (ms vs ns).
3. Frame penuh $\rightarrow$ pilih *victim* dengan algoritma.
4. **FIFO**: sederhana, rentan Belady's anomaly.
5. **OPT**: ideal tapi tidak bisa diimplementasikan.
6. **LRU**: praktis dan mendekati optimal.

Checklist Keterampilan

Pastikan Anda dapat melakukan ini

TLB & EAT:

- ☐ Menjelaskan perbedaan TLB hit dan TLB miss.
- ☐ Menghitung T_{hit} dan T_{miss} dari parameter yang diberikan.
- ☐ Menghitung EAT dari hit rate dan waktu akses.
- ☐ Menjelaskan peran ASID saat context switch.

Demand Paging & Page Replacement:

- ☐ Membaca page table dan menentukan page hit/fault.
- ☐ Menjelaskan 6 langkah penanganan page fault oleh OS.
- ☐ Men-trace FIFO, OPT, dan LRU step-by-step.
- ☐ Membandingkan jumlah page fault ketiga algoritma.

Ada Pertanyaan?

Jika ada satu konsep yang belum jelas, jangan lanjut.
Tanyakan sekarang atau diskusikan di forum kuliah.

*“The art of doing mathematics consists in finding that special case
which contains all the germs of generality.”*

— David Hilbert